RLHF → RLVR: Feedback Signals for Post-Training and Acting LLMs

Bennet Brown bnb2122@uw.edu
Nikhil Dhanda nidhanda@uw.edu
Rupesh Patro patro@uw.edu
Yaoxian Qu (yaoxiq) quyaoxian@gmail.com

Ying Zheng yzheng82@cs.washington.edu

Abstract

Modern large language model (LLM) post-training can be understood through the lens of feedback: what signal is available, how reliable it is, and how it should be used to shape behavior. Classical reinforcement learning from human feedback (RLHF) answers this question with a three-stage pipeline: supervised fine-tuning (SFT) on demonstrations, reward modeling from human preferences, and policy optimization with methods such as Proximal Policy Optimization (PPO) [Ouyang et al., 2022, Schulman et al., 2017]. More recent work has pushed in two adjacent directions. First, direct preference methods such as Direct Preference Optimization (DPO), Odds Ratio Preference Optimization (ORPO), and Kahneman-Tversky Optimization (KTO) simplify alignment by optimizing directly on preference data without an explicit reward model or online RL loop [Rafailov et al., 2023, Hong et al., 2024, Ethayarajh et al., 2024]. Second, reinforcement learning with verifiable rewards (RLVR) replaces learned preference signals with correctness checks, tests, or tool success, making reward computation cheaper and more scalable in reasoning and agentic settings [Shao et al., 2024, Guo et al., 2025]. This literature review organizes the field around feedback sources and optimization mechanisms. It argues that the movement from RLHF to RLVR is best understood not as a wholesale replacement of one paradigm by another, but as a reorganization of post-training around feedback availability, evaluation reliability, and deployment context.

1 Introduction

Post-training is the stage at which a pretrained language model becomes a usable assistant, reasoner, or agent. In the now-canonical RLHF pipeline, a pretrained model is first instruction-tuned on curated demonstrations, then a reward model is trained from human preference comparisons, and finally the policy is optimized with reinforcement learning while remaining close to a reference model [Ouyang et al., 2022]. This recipe was foundational because it showed that alignment depends not only on model scale, but also on how post-training supervision is structured. InstructGPT, for example, demonstrated that a 1.3B aligned model could be preferred over a much larger 175B base model, underscoring the importance of post-training as the stage that makes language models usable rather than merely capable [Ouyang et al., 2022].

Yet RLHF also has clear bottlenecks. Preference data is expensive to collect, reward models can inherit the biases and blind spots of their labels, and PPO-style online RL is often fragile and expensive to tune [Stiennon et al., 2020, Ouyang et al., 2022, Huang et al., 2024]. In response, the field has expanded in two directions. One direction simplifies the optimizer and removes the explicit reward-model-plus-PPO stack, as in DPO-family methods [Rafailov et al., 2023, Hong et al., 2024, Ethayarajh et al., 2024]. The other direction replaces learned rewards with verifiable ones, especially

Preprint. Under review.

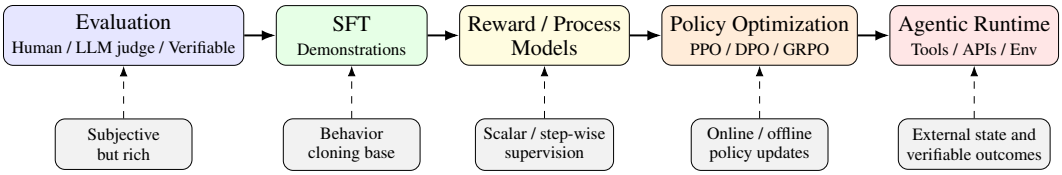

Figure 1: Overview of the post-training pipeline organized by feedback type: evaluation, SFT, reward/process models, policy optimization, and agentic runtime. The central thesis of this review is that post-training methods are best understood by the kind of feedback they can exploit.

in reasoning and tool-using domains where correctness can be checked by external systems [Shao et al., 2024, Guo et al., 2025, Jimenez et al., 2023]. Across both directions, evaluation remains central, because any claimed gain in alignment is only as meaningful as the metric that certifies it [Zheng et al., 2023, Dubois et al., 2024, Zhou et al., 2023b].

This review synthesizes the literature under a single guiding question: what is the feedback signal, how is it computed, and what failure modes does it introduce? The answer unfolds in five stages. First, evaluation defines what counts as progress and reveals where common metrics fail. Second, SFT establishes the behavior-cloning base that most later methods build on. Third, reward and process models determine where scalar or step-level supervision comes from. Fourth, preference optimization and RLVR specify how the policy is updated. Finally, agentic runtimes show where verifiable feedback becomes operationally natural, because tools, APIs, and environments provide external state transitions and success criteria that can be checked directly.

2 Evaluation and Reliability

Evaluation is not an afterthought in modern LLM post-training; it determines what kinds of improvements can even be observed. The literature surveyed here points to a three-part evaluation landscape: human evaluation, model-based auto-evaluation, and verifiable benchmarks. Human evaluation remains the standard for subjective qualities such as helpfulness, style, or conversational tone, but it is expensive and difficult to reproduce. Model-based evaluation scales more easily, yet it introduces its own biases. Verifiable evaluation provides the strongest objectivity, but only in domains where constraints or outcomes can be checked automatically. The practical lesson is that no single evaluation mode is sufficient on its own [Zheng et al., 2023, Dubois et al., 2024, Zhou et al., 2023b].

Human evaluation is most visible in systems such as Chatbot Arena, where models are compared through pairwise judgments aggregated with Elo-style rating systems [Zheng et al., 2023]. This framework is valuable because it converts subjective human preference into a quantitative ranking, but it remains costly and noisy. Arena-style evaluation is particularly useful for open-ended conversational systems, where there may be many acceptable responses and no single deterministic answer. Its weakness is precisely that it depends on subjective judgment and repeated human annotation. This makes it difficult to scale and hard to reproduce as a fully objective benchmark.

Model-based evaluation attempts to reduce this burden by prompting another LLM to act as a judge or by using likelihood-based signals as proxies for preference. The appeal is obvious: model judges are cheap, fast, and reusable across many tasks. However, Zheng et al. [2023] show that LLM-as-a-judge systems exhibit position bias, verbosity bias, and self-enhancement bias. Dubois et al. [2024] make a closely related point in Length-Controlled AlpacaEval, where even a seemingly simple auto-evaluator can be confused by response length. These findings matter because automated evaluation is now used not only for benchmarking but increasingly for development iteration. A biased evaluator can lead to a biased training loop.

Verifiable benchmarks therefore occupy a special place in this survey. IFEval was designed precisely to avoid subjective judgment by focusing on instruction-following constraints that can be checked algorithmically, such as formatting, length, or explicit keyword requirements [Zhou et al., 2023b]. RewardBench brings the same spirit to reward model evaluation by testing whether preference models actually distinguish chosen from rejected responses on challenging examples, rather than merely overfitting surface style [Lambert et al., 2024]. SWE-bench pushes verifiable evaluation into software engineering: the model is given a real GitHub issue and succeeds only if its code edits resolve the

issue under test execution [Jimenez et al., 2023]. Together, these benchmarks suggest a broader shift in alignment research: where possible, the field increasingly prefers metrics grounded in checkable outcomes over ones grounded only in subjective impressions.

The broader reliability lesson is that evaluation and training are converging. When an evaluation metric is verifiable and closely tied to downstream success, it often becomes an appealing reward signal for RLVR. This is why benchmarks such as IFEval, RewardBench, ToolBench, τ-bench, and SWE-bench matter not only as diagnostics but also as signs of where scalable post-training is heading.

3 Foundations: Supervised Fine-Tuning and Instruction Tuning

Supervised fine-tuning is the behavioral base of nearly all modern post-training pipelines. InstructGPT established the modern template: collect prompt-response demonstrations from human labelers, fine-tune a pretrained language model on these demonstrations, and thereby convert a general text completer into an instruction follower [Ouyang et al., 2022]. This stage is analogous to behavior cloning in reinforcement learning: instead of learning from reward and exploration, the model imitates high-quality expert trajectories. InstructGPT showed that this alone substantially improved user preference, truthfulness, and toxicity outcomes relative to the base GPT-3 model.

But the same work also revealed SFT's limitations. Because supervised datasets contain only positive demonstrations, they rarely express comparative information: the model learns what one good answer looks like, but not why it is better than alternatives. It therefore struggles to represent tradeoffs such as helpfulness versus harmlessness or verbosity versus precision. This limitation motivates later preference-based and verifier-based methods, which add explicit signals about relative quality or correctness rather than merely imitating demonstrations.

Finetuned Language Models (FLAN) broadened the SFT story by showing that instruction tuning can improve zero-shot performance across many tasks when a pretrained model is fine-tuned on a large, instruction-formatted mixture of datasets [Wei et al., 2022]. The key result is not just better performance on seen tasks, but transfer to unseen task clusters, suggesting that instruction tuning teaches a reusable meta-skill: parsing and responding appropriately to natural language instructions. In the logic of this literature review, FLAN provides early evidence that SFT is not merely a formatting layer, but a general-purpose alignment stage that unlocks latent capabilities in pretrained models.

Self-Instruct and Stanford Alpaca addressed the cost bottleneck of collecting large instruction datasets by generating synthetic instruction-response pairs from strong teacher models [Wang et al., 2023, Taori et al., 2023]. Self-Instruct demonstrates that instruction-following data can be bootstrapped from a small human seed set, while Alpaca shows how this recipe can be reproduced cheaply with open models and commercial APIs. Their shared contribution is economic as much as algorithmic: they lowered the barrier to entry for instruction tuning. Their shared weakness is that synthetic supervision inherits the limitations of its generator, including stylistic narrowness, factual errors, and hidden biases.

Less Is More for Alignment (LIMA) sharpened the quality-versus-quantity debate by arguing that a small set of carefully curated demonstrations can rival or exceed much larger synthetic instruction datasets [Zhou et al., 2023a]. Its "less is more" result suggests that much of alignment through SFT may operate at the level of output format and behavioral style rather than deeply changing capabilities acquired in pretraining. Tulu 3 then synthesizes these lessons into a modern open post-training recipe, treating data curation as a multi-objective optimization problem across capabilities such as reasoning, safety, multilinguality, and precise instruction following [Ivison et al., 2024a]. Crucially, Tulu 3 also demonstrates through ablations that SFT captures a large fraction of final performance, but not all of it. Later preference tuning and RLVR still produce meaningful gains. Thus, the strongest conclusion from the SFT literature is not that SFT is insufficient and therefore secondary, but rather that it is indispensable and therefore foundational.

4 Reward Models, AI Feedback, and Process/Verifier Models

If SFT teaches the model how to respond, reward modeling determines how responses should be scored. A standard reward model is trained on pairwise preference data, learning to assign higher scalar values to preferred responses than to rejected ones. *Learning to Summarize from Human*

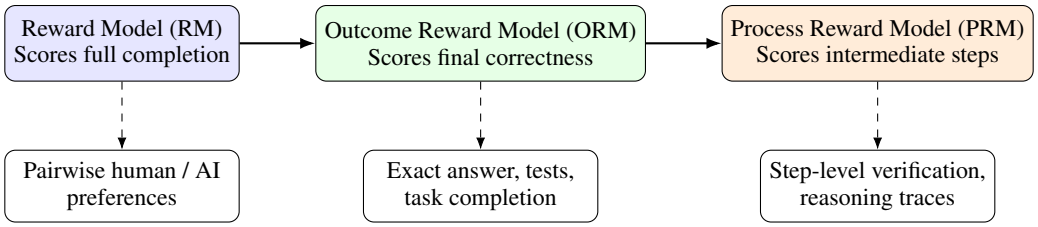

Figure 2: Comparison of reward models, outcome reward models, and process reward models by what they score and how dense or verifiable the supervision is. The progression helps explain the conceptual bridge from classical RLHF toward RLVR.

Feedback is the canonical example: human annotators compare alternative summaries, and a reward model is trained with a Bradley–Terry style objective so that preferred summaries receive higher scores [Stiennon et al., 2020, Bradley and Terry, 1952]. The output of this model becomes the reward signal for downstream policy optimization.

The reward-modeling literature emphasizes that data quality is a central design variable. Stiennon et al. [2020] devote substantial effort to labeler alignment and consistency, highlighting a point that remains important across later work: a reward model is only as good as the comparisons it learns from. If labels reflect superficial preferences, dataset artifacts, or inconsistent standards, the reward model will reproduce those flaws and the policy will optimize them. This concern is one reason reward evaluation itself has become an active area, as in RewardBench [Lambert et al., 2024].

A major extension of this pipeline is the move from human feedback to AI feedback. RLAIF vs. RLHF shows that strong LLMs can generate preference labels that are competitive with human annotations on some tasks, significantly reducing the cost of reward model training [Lee et al., 2024]. Constitutional AI goes a step further by incorporating explicit natural-language principles into the feedback process, especially for harmlessness, so that AI systems critique and revise outputs according to a constitution rather than relying solely on direct human labels [Bai et al., 2022b]. In both cases, the downstream RL loop remains recognizable, but the source of supervision changes. The important conceptual shift is that reward is no longer tied exclusively to direct human preference; it can also be mediated through model judgments or principle-guided AI judgments.

The literature then differentiates among several kinds of reward models. Standard reward models score full completions using pairwise preferences. Outcome reward models score only final-answer correctness, often in domains where correctness is verifiable. Process reward models (PRMs) go further by assigning supervision to intermediate reasoning steps rather than only to final outputs. *Let's Verify Step by Step* is the central reference here: it trains a verifier on step-level annotations over mathematical reasoning traces, allowing the model to assess each step in a chain of thought rather than only the final answer [Lightman et al., 2023]. This denser supervision is useful not only for training but also for search, reranking, and test-time verification. More recent work such as ThinkPRM pushes this direction further by aiming for more data-efficient, verifier-style process supervision [Khalifa et al., 2025].

This evolution from reward models to PRMs is an important bridge between RLHF and RLVR. Classical RLHF works with relatively sparse scalar rewards, typically attached to full generations. Process supervision moves reward inward, from outcomes toward trajectories. Once this shift happens, it becomes natural to imagine training and evaluation in environments where intermediate actions, tool calls, and state transitions all provide verifiable feedback.

5 Preference Optimization and RLVR Policy Optimization

Once demonstrations and reward signals have been defined, the remaining question is algorithmic: how should the policy itself be updated? The literature considered here suggests three broad answers. The first is classical online RLHF, in which a policy is sampled, scored by a reward model, and updated with policy-gradient methods such as PPO [Ouyang et al., 2022, Schulman et al., 2017]. The second is direct preference optimization, which bypasses explicit reward-model training and online RL by optimizing directly on preference pairs [Rafailov et al., 2023, Hong et al., 2024, Ethayarajh

et al., 2024]. The third is RLVR, in which the reward comes not from a learned reward model but from an external verifier such as exact-answer checking, theorem verification, test execution, or tool success [Shao et al., 2024, Guo et al., 2025].

The online RLHF story begins conceptually with sequence-level policy gradients, but in practice quickly converges on trust-region style optimization. TRPO introduced the general principle of limiting policy updates, and PPO made this principle practical through clipped surrogate objectives that support multiple minibatch updates without excessively large shifts in policy behavior [Schulman et al., 2015, 2017]. In LLM alignment, InstructGPT became the canonical realization of this idea: reward is shaped by a Kullback-Leibler (KL) penalty relative to a reference model, and the policy is optimized to improve reward while remaining close to its SFT initialization [Ouyang et al., 2022]. Later reproduction work makes clear that this recipe is much more sensitive in practice than it appears on paper, with reward normalization, KL handling, sampling policy, and optimizer configuration all playing major roles [Huang et al., 2024].

RLVR preserves the online RL structure but changes the source of reward. Instead of fitting a reward model to preference data, it uses a reward that can be checked automatically. DeepSeekMath is the key example in the survey: it introduces Group Relative Policy Optimization (GRPO), a PPO-style method that dispenses with a separate critic and instead estimates a baseline from a group of sampled completions, reducing memory cost while retaining a relative advantage signal [Shao et al., 2024]. DeepSeek-R1 extends this paradigm to large-scale reasoning, arguing that strong reasoning behavior can emerge from reinforcement learning driven primarily by verifiable rewards rather than human-labeled traces [Guo et al., 2025]. In this view, reasoning domains such as mathematics and coding become especially attractive because external checkers can provide cheap and unambiguous signals.

Recent follow-up work shows that GRPO is not the end of the RLVR story but the start of a new optimizer design space. DAPO focuses on long-chain reasoning RL and proposes changes aimed at reducing entropy collapse and instability, including decoupled clipping, dynamic sampling, token-level policy-gradient losses, and reward shaping for overlong outputs. Understanding R1-Zero-Like Training argues that the original GRPO objective is length-biased and introduces Dr. GRPO to improve token efficiency while preserving performance. Revisiting GRPO then studies both on-policy and off-policy variants, suggesting that off-policy GRPO can sometimes match or exceed on-policy training in RLVR settings [Liu et al., 2025, Mroueh et al., 2025]. Taken together, these papers suggest that once reward becomes verifiable, optimization remains the primary difficulty: sample efficiency, stability, and vulnerability to reward hacking do not disappear simply because the reward is easier to compute.

In parallel, direct alignment methods ask whether online RL can be avoided altogether. DPO shows that a KL-constrained RLHF objective can be rewritten as a classification-style objective over chosen and rejected responses, thereby eliminating the need for explicit reward-model training and online rollout collection [Rafailov et al., 2023]. ORPO extends this family by removing the explicit reference model and integrating preference optimization into a monolithic objective [Hong et al., 2024]. KTO broadens the setup further by learning from binary desirable/undesirable signals using a prospect-theoretic framing of human utility [Ethayarajh et al., 2024]. These methods are best seen as offline direct-alignment algorithms rather than as standard off-policy RL, because their appeal lies primarily in simplicity and throughput.

However, the literature surveyed here is careful not to equate simplicity with superiority. Unpacking DPO and PPO shows that when data, prompts, and reward-model choices are carefully controlled, PPO can outperform DPO in some settings [Ivison et al., 2024b]. Understanding the performance gap between online and offline alignment algorithms makes a complementary argument: on-policy sampling appears to improve generation behavior in ways that static pairwise classification does not fully capture [Tang et al., 2024]. At the same time, Back to Basics shows that simpler online methods—carefully adapted REINFORCE-style variants—can outperform both PPO and several "RL-free" methods in some alignment settings [Ahmadian et al., 2024]. The resulting picture is not one of a single best optimizer, but of a tradeoff shaped by feedback regime. PPO-style RLHF remains strong when reward models and online sampling are affordable; DPO-family methods are compelling when only static preference data is available; RLVR methods are especially attractive when correctness can be checked cheaply and at scale.

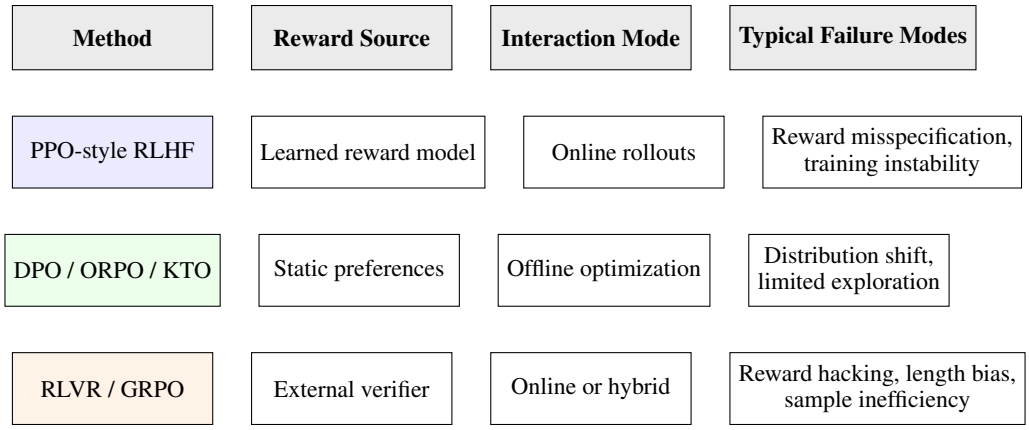

Figure 3: Comparison of PPO-style RLHF, DPO-family methods, and RLVR/GRPO by reward source, interaction regime, and major failure modes. This summary emphasizes that the optimizer choice depends on the feedback regime rather than a single universal ordering of methods.

6 Agentic Runtime and Orchestration

The final step in this literature review moves from post-training in the abstract to agents acting in environments. This is where the connection between RLVR and deployment becomes most concrete. In ordinary chat use, model quality is often subjective and difficult to verify. In agentic settings, however, the model takes actions in the world through tools or APIs, and those actions produce state changes that can often be checked directly. Tool success, failed execution, or final environment state therefore become natural reward and evaluation signals.

ReAct is the simplest and most influential framing of this idea. It interleaves reasoning traces and actions, allowing the model to plan, act, observe, and continue reasoning in a loop [Yao et al., 2022]. The importance of ReAct is conceptual as much as empirical: it bridges the gap between language-only reasoning and action-conditioned problem solving. Once reasoning and acting are interleaved, feedback no longer comes only from textual preference. It also comes from the environment.

Toolformer shifts tool use from prompting-time orchestration to training-time learning. Instead of relying entirely on hand-written few-shot demonstrations, it constructs a training corpus in which tool calls are inserted when they reduce language-modeling loss, thereby teaching the model when and how to use external tools in a self-supervised manner [Schick et al., 2023]. Gorilla addresses a different problem: API calls fail when documentation changes, arguments are renamed, or tool interfaces evolve. Its retriever-aware setup teaches the model to condition on current documentation, improving robustness to shifting APIs and emphasizing that reliable tool use depends not only on a model's internal knowledge but also on its ability to retrieve and interpret external tool descriptions at inference time [Patil et al., 2023].

ToolBench and τ-bench push the field toward richer evaluations of agentic behavior. ToolBench is both a benchmark and a practical recipe, emphasizing that open models often require explicit examples, in-context demonstrations, and output-style constraints in order to use tools reliably [Xu et al., 2023]. τ-bench then moves beyond single-turn tool calls toward full user-agent-tool interaction. Its state-based verification setup evaluates whether the final database or environment state matches a target goal, and its pass^k metric emphasizes repeatability rather than average one-shot success [Yao et al., 2024]. This is a crucial development: for practical agents, consistency and reliability matter as much as raw average performance.

SWE-bench occupies a special role because it turns software engineering into a realistic, verifiable agentic environment. The model is given a real repository and a GitHub issue, and succeeds only if it modifies the code so that the issue is resolved under test execution [Jimenez et al., 2023]. This setup strongly encourages an agentic architecture involving repository inspection, planning, multi-file editing, execution, and feedback loops. In that sense, SWE-bench is more than an evaluation benchmark; it is also an argument for why agentic reasoning and RLVR belong together. Once

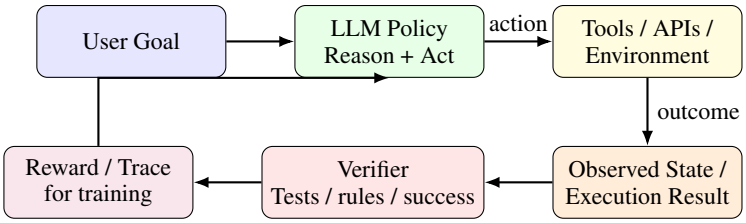

Figure 4: Agentic runtime as a natural RLVR setting. Once an LLM acts through tools or APIs, external state transitions and testable outcomes provide direct feedback for evaluation and potentially for learning.

the environment itself can verify success, reward can be grounded in external state rather than in subjective textual judgment.

7 Discussion: From RLHF to RLVR as a Shift in Feedback Availability

Across these literatures, a common structure emerges. SFT provides the initial behavioral scaffold. Reward models and AI feedback define how preferences can be turned into scalar supervision. PPO-style RLHF offers one way to optimize against that supervision online. Direct alignment methods simplify the same problem when only static preference data is available. RLVR becomes attractive when the task itself offers a cheap verifier. Agentic runtimes are where such verifiers often appear naturally, because actions have external consequences that can be checked.

Seen this way, the shift from RLHF to RLVR is less a paradigm replacement than a change in what kinds of feedback researchers are able to exploit. Early alignment work was bottlenecked by human labels and preference models. More recent work increasingly exploits AI judges, process supervision, exact-answer checkers, tests, and environment states. This shift changes not only the economics of post-training, but also the optimizer choices that make sense. Dense but noisy preference data favors one class of methods; sparse but verifiable rewards favor another.

A useful practitioner-oriented guide follows directly from this taxonomy. When the available signal is a set of high-quality demonstrations, SFT is the correct starting point. When there are pairwise preferences but limited online RL infrastructure, DPO-family methods offer a practical direct-alignment path. When reward models can be trained reliably and online exploration is affordable, PPO-style RLHF remains a strong option. When correctness can be verified externally through math checkers, tests, or state transitions, RLVR becomes especially attractive. And when the task is inherently interactive and tool-based, the training and evaluation stack should be designed around the runtime itself, since environment interaction provides the clearest path to verifiable feedback.

8 Conclusion

The literature on post-training is often narrated as a sequence of replacements: SFT gave way to RLHF, RLHF gave way to DPO, and DPO is now being overtaken by RLVR. This survey suggests a different view. Modern post-training is best understood as a layered design space in which different methods become attractive under different feedback regimes. SFT remains the indispensable behavioral base. Reward models and AI feedback expand the range of supervision available beyond raw demonstrations. Preference optimization simplifies alignment when preference data is static. RLVR takes advantage of settings where correctness is verifiable. Agentic tool use makes such settings increasingly common.

The most important consequence of this perspective is practical. The central question in post-training is not simply which optimizer is newest or which benchmark score is highest. It is what signal the system can trust. The history from RLHF to RLVR is, at its core, a history of turning weaker, more expensive, and more subjective forms of feedback into stronger, cheaper, and more verifiable ones.

Table 1: Decision guide mapping feedback availability and practical constraints to major post-training approaches.

Feedback available	Typical setting	Best starting point	Why
High-quality demonstrations	Instruction following, style shaping, assistant behavior	SFT	Strongest when imitation targets are available and task success is not easily verifiable.
Static pairwise preferences	Preference datasets without online rollout infrastructure	DPO / ORPO / KTO	Avoids reward-model training and expensive online RL.
Reliable learned reward model + rollout budget	Mature RLHF stack with human or AI preference labels	PPO-style RLHF	Benefits from on-policy updates and explicit reward optimization.
Externally checkable correctness	Math, coding, theorem proving, execution-based tasks	RLVR / GRPO-family	Cheap, scalable, and objective reward computation.
Interactive tool use with state transitions	Agents using APIs, software tools, databases, browsers	Agentic training + RLVR	Runtime produces natural verification signals through environment outcomes.

References

Arian Ahmadian, Chris Cremer, Matthias Gallé, Marzieh Fadaee, Julia Kreutzer, Olivier Pietquin, Ahmet Üstün, and Sara Hooker. Back to Basics: Revisiting REINFORCE Style Optimization for Learning from Human Feedback in LLMs. In *ACL*, 2024.

Yuntao Bai, Andy Jones, Kamal Ndousse, Amanda Askell, Anna Chen, Nova DasSarma, Dawn Drain, Stanislav Fort, Deep Ganguli, Tom Henighan, et al. Training a Helpful and Harmless Assistant with Reinforcement Learning from Human Feedback. *arXiv preprint arXiv:2204.05862*, 2022.

Yuntao Bai, Saurav Kadavath, Sandipan Kundu, Amanda Askell, Jackson Kernion, Andy Jones, Anna Chen, Anna Goldie, Azalia Mirhoseini, Cameron McKinnon, et al. Constitutional AI: Harmlessness from AI Feedback. *arXiv preprint arXiv:2212.08073*, 2022.

Ralph A. Bradley and Milton E. Terry. Rank Analysis of Incomplete Block Designs: I. The Method of Paired Comparisons. *Biometrika*, 39(3/4): 324–345, 1952.

Yann Dubois, Balázs Galambosi, Percy Liang, and Tatsunori B. Hashimoto. Length-Controlled AlpacaEval: A Simple Way to Debias Automatic Evaluators. *arXiv preprint arXiv:2404.04475*, 2024.

Kawin Ethayarajh, Winnie Xu, Niklas Muennighoff, Dan Jurafsky, and Douwe Kiela. KTO: Model Alignment as Prospect Theoretic Optimization. *arXiv preprint arXiv:2402.01306*, 2024.

Daya Guo, Dejian Yang, Haowei Zhang, et al. DeepSeek-R1: Incentivizing Reasoning Capability in LLMs via Reinforcement Learning. *arXiv preprint arXiv:2501.12948*, 2025.

Jiwoo Hong, Noah Lee, and James Thorne. ORPO: Monolithic Preference Optimization without Reference Model. *arXiv preprint arXiv:2403.07691*, 2024.

Shengyi Huang, Michael Noukhovitch, Aref Hosseini, Kashif Rasul, Weizhe Wang, and Lewis Tunstall. The N+ Implementation Details of RLHF with PPO: A Case Study on TL;DR Summarization. *arXiv preprint arXiv:2403.17031*, 2024.

Hamish Ivison, Nathan Lambert, Jacob Morrison, Valentina Pyatkin, Shengyi Huang, Faeze Brahman, Lester James V. Miranda, Alisa Liu, Nouha Dziri, Shanshan Lyu, et al. Tulu 3: Pushing Frontiers in Open Language Model Post-Training. *arXiv preprint arXiv:2411.15124*, 2024.

Hamish Ivison, Yizhong Wang, Jingwei Liu, Zexuan Wu, Valentina Pyatkin, Nathan Lambert, Noah A. Smith, Yejin Choi, and Hannaneh Hajishirzi. Unpacking DPO and PPO: Disentangling Best Practices for Learning from Preference Feedback. *arXiv preprint arXiv:2406.09279*, 2024.

Carlos E. Jimenez, John Yang, Alexander Wettig, Shunyu Yao, Kexin Pei, Ofir Press, and Karthik Narasimhan. SWE-bench: Can Language Models Resolve Real-World GitHub Issues? *arXiv preprint arXiv:2310.06770*, 2023.

Muhammad Khalifa et al. Process Reward Models That Think. *arXiv preprint arXiv:2504.16828*, 2025.

Nathan Lambert, Valentina Pyatkin, Jacob Morrison, Lester J. Miranda, Bill Y. Lin, Khyathi Chandu, Nouha Dziri, Sachin Kumar, Tom Zick, Yejin Choi, Noah A. Smith, and Hannaneh Hajishirzi. RewardBench: Evaluating Reward Models for Language Modeling. *arXiv preprint arXiv:2403.13787*, 2024.

Hyung Won Lee, Shreyas Phatale, Hamza Mansoor, Thomas Mesnard, Johan Ferret, Kai Lu, Chris Bishop, Emily Hall, Victor Carbune, Abhinav Rastogi, and Shashank Prakash. RLAIF vs. RLHF: Scaling Reinforcement Learning from Human Feedback with AI Feedback. In *ICML*, 2024.

Hunter Lightman, Vineet Kosaraju, Yuri Burda, Harri Edwards, Bowen Baker, Teddy Lee, Jan Leike, John Schulman, Ilya Sutskever, and Karl Cobbe. Let's Verify Step by Step. *arXiv preprint arXiv:2305.20050*, 2023.

Zeyu Liu, Cheng Chen, Wenhao Li, Peiwen Qi, Tianyu Pang, Chao Du, Wee Sun Lee, and Min Lin. Understanding R1-Zero-Like Training: A Critical Perspective. *arXiv preprint arXiv:2503.20783*, 2025.

Youssef Mroueh, Nicolas Dupuis, Bastien Belgodere, Aniket Nitsure, Mattia Rigotti, Kristjan Greenewald, Jiri Navratil, Jack Ross, and Joseph Rios. Revisiting Group Relative Policy Optimization: Insights into On-Policy and Off-Policy Training. *arXiv preprint arXiv:2505.22257*, 2025.

Long Ouyang, Jeffrey Wu, Xu Jiang, Diogo Almeida, Carroll L. Wainwright, Pamela Mishkin, Chong Zhang, Sandhini Agarwal, Katarina Slama, Alex Ray, et al. Training Language Models to Follow Instructions with Human Feedback. In *Advances in Neural Information Processing Systems*, 2022.

Shishir G. Patil, Tianjun Zhang, Xuezhi Wang, and Joseph E. Gonzalez. Gorilla: Large Language Model Connected with Massive APIs. *arXiv preprint arXiv:2305.15334*, 2023.

Rafael Rafailov, Archit Sharma, Eric Mitchell, Stefano Ermon, Christopher D. Manning, and Chelsea Finn. Direct Preference Optimization: Your Language Model is Secretly a Reward Model. In *Advances in Neural Information Processing Systems*, 2023.

Timo Schick, Jane Dwivedi-Yu, Roberto Dessì, Roberta Raileanu, Maria Lomeli, Luke Zettlemoyer, Nicola Cancedda, and Thomas Scialom. Toolformer: Language Models Can Teach Themselves to Use Tools. In *Advances in Neural Information Processing Systems*, 2023.

John Schulman, Sergey Levine, Philipp Moritz, Michael Jordan, and Pieter Abbeel. Trust Region Policy Optimization. In *ICML*, 2015.

John Schulman, Filip Wolski, Prafulla Dhariwal, Alec Radford, and Oleg Klimov. Proximal Policy Optimization Algorithms. *arXiv preprint arXiv:1707.06347*, 2017.

Zhuosheng Shao, Peiyi Wang, Qi Zhu, Runxin Xu, Junxiao Song, Xiaodong Bi, Haoran Zhang, Mingchuan Zhang, Yankai Li, Yuxiang Wu, and Daya Guo. DeepSeekMath: Pushing the Limits of Mathematical Reasoning in Open Language Models. *arXiv preprint arXiv:2402.03300*, 2024.

Nisan Stiennon, Long Ouyang, Jeffrey Wu, Daniel M. Ziegler, Ryan Lowe, Chelsea Voss, Alec Radford, Dario Amodei, and Paul Christiano. Learning to Summarize from Human Feedback. In *Advances in Neural Information Processing Systems*, 2020.

Yunhao Tang, D. Z. Guo, Zhi Zheng, Daniele Calandriello, Yongqi Cao, Evgenii Tarassov, Rémi Munos, Bernardo Ávila Pires, Michal Valko, Yutian Cheng, and Will Dabney. Understanding the Performance Gap between Online and Offline Alignment Algorithms. *arXiv preprint arXiv:2405.08448*, 2024.

Rohan Taori, Ishaan Gulrajani, Tianyi Zhang, Yann Dubois, Xuechen Li, Carlos Guestrin, Percy Liang, and Tatsunori B. Hashimoto. Alpaca: A Strong, Replicable Instruction-Following Model. Stanford Center for Research on Foundation Models, 2023.

Yizhong Wang, Yeganeh Kordi, Swaroop Mishra, Alisa Liu, Noah A. Smith, Daniel Khashabi, and Hannaneh Hajishirzi. Self-Instruct: Aligning Language Models with Self-Generated Instructions. In *ACL*, 2023.

Jason Wei, Maarten Bosma, Vincent Y. Zhao, Kelvin Guu, Adams Wei Yu, Brian Lester, Nan Du, Andrew M. Dai, and Quoc V. Le. Finetuned Language Models Are Zero-Shot Learners. In *ICLR*, 2022.

Qiancheng Xu, Fanjin Hong, Biao Li, Chenglin Hu, Zhenyu Chen, and Jingren Zhang. On the Tool Manipulation Capability of Open-Source Large Language Models. *arXiv preprint arXiv:2305.16504*, 2023.

Shunyu Yao, Jeffrey Zhao, Dian Yu, Nan Du, Izhak Shafran, Karthik Narasimhan, and Yuan Cao. ReAct: Synergizing Reasoning and Acting in Language Models. *arXiv preprint arXiv:2210.03629*, 2022.

Shunyu Yao, Noah Shinn, Parisa Razavi, and Karthik Narasimhan. τ-bench: A Benchmark for Tool-Agent-User Interaction in Real-World Domains. *arXiv preprint arXiv:2406.12045*, 2024.

Lianmin Zheng, Wei-Lin Chiang, Ying Sheng, Siyuan Zhuang, Ziwen Wu, Yonghao Zhuang, Zhuohan Lin, Zhe Li, Dacheng Li, Eric P. Xing, Hao Zhang, Joseph E. Gonzalez, and Ion Stoica. Judging LLM-as-a-Judge with MT-Bench and Chatbot Arena. In *Advances in Neural Information Processing Systems*, 2023.

Chunting Zhou, Pengfei Liu, Pinyu Xu, Srinivasan Iyer, Jianfei Sun, Yichao Mao, Xuezhe Ma, Alon Efrat, Ping Yu, Lili Yu, Shiyu Zhang, Gourab Ghosh, Mike Lewis, Luke Zettlemoyer, and Omer Levy. LIMA: Less Is More for Alignment. In *Advances in Neural Information Processing Systems*, 2023.

Jeffrey Zhou, Tian Lu, Swaroop Mishra, Siddhartha Brahma, Sreyan Basu, Yi Luan, Denny Zhou, and Le Hou. Instruction-Following Evaluation for Large Language Models. *arXiv preprint arXiv:2311.07911*, 2023.